JFrog | CONAN C/C++ Package Manager

ROSCon España 2026 · Valencia

ROS sin rodeos

Desarrollo nativo en C++ para robótica

Daniel Manzaneque
Ingeniero Software · Conan / JFrog



conan.io github.com/conan-io/ros-conan

De dónde venimos...

Un robot pequeño con un ordenador modesto

- Una placa con pocos recursos (tipo Raspberry Pi)
- Control y percepción escritos en **C++**
- **ROS 2** para hablar con sensores, herramientas y otros equipos

El punto de partida

«Solo quería publicar un topic desde mi código C++»

Antes de escribir una línea útil, hay que:

1. Tener la versión de Ubuntu que pide la distro
2. Instalar con `apt` y `rosdep` todo lo demás
3. Crear un workspace de colcon con su estructura
4. Escribir `package.xml` y adaptar el CMake a `ament_cmake`
5. Acordarse de hacer `source setup.bash`
6. ...

Muchos pasos antes de llegar al robot

El mapa de la charla

Tres rodeos mejorables

01

Instalar

ROS se instala en el sistema → ROS pasa a ser **una dependencia más**.

02

Desarrollar

Tu proyecto se adapta a ROS → Tu **proyecto C++ de siempre**.

03

Desplegar

Compilar en el robot o en la imagen de despliegue → **Construir fuera**, llevar solo lo necesario.

La herramienta

Conan, el gestor de paquetes open source de C y C++

Código abierto (MIT)

- Mantenido por el equipo de Conan con el feedback de la comunidad

Cada librería es una receta

- Hay unas 2.000 recetas en ConanCenter y puedes crear las tuyas.

Agnóstico al build-system

- Compatible con CMake, Bazel, Meson, MSBuild...

Resuelve el grafo completo de dependencias transitivas

- Obtiene un **binario por configuración**, si no está disponible lo compila desde el código fuente.



Una versión por sistema. Una imagen por distro

- `apt` y `rosdep`: instalan la versión que trae la distro. `package.xml` declara dependencias, pero no elige versiones
- **Docker**: congela esa foto y la hace repetible. Otra distro de ROS, otra imagen
- ¿Otra versión de una librería? La compilas a mano y nadie comprueba que no choque con la de ROS
- Y el desarrollo se muda dentro de la imagen, lejos de tu IDE

> `apt` y Docker resuelven bien la instalación. Lo que no ven es el grafo de tu proyecto.

Sin rodeos

ROS es una dependencia más

conanfile.txt

```
[requires]
ros-kilted/2026.06.17
opencv/4.12.0
```

- ROS vive en la **caché de Conan**, no en `/opt/ros` ni en una imagen
- Varias distros de ROS en la misma máquina, **sin una imagen por distro**

```
conan install conanfile.txt --build=missing
```

- El mismo flujo en **Linux, macOS y Windows**

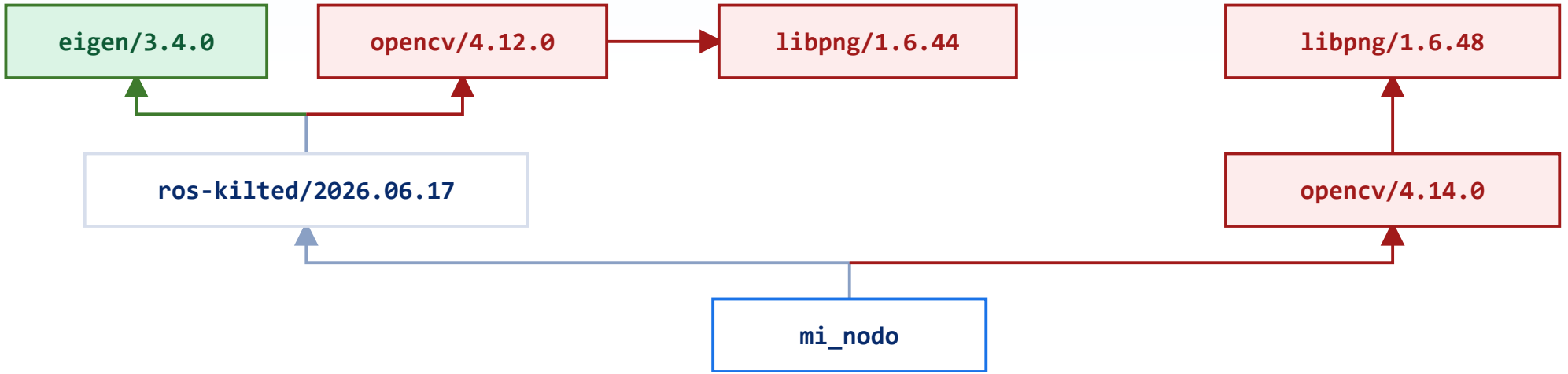
El paquete

¿Qué es `ros-kilted`?

- **ROS 2 Kilted compilado desde fuentes**, empaquetado como un único paquete Conan
- **Versión = sincronización de rosdistro**: `2026.06.17` es el tag `kilted/2026-06-17`
- **Variantes** como los metapaquetes oficiales: `core`, `base`, `desktop` `-o ros-kilted/*:variant=desktop`
- Las **dependencias de terceros** (OpenSSL, Eigen, yaml-cpp, OpenCV...) vienen de ConanCenter: son nodos del grafo
- Incluye `ros2` y `colcon`: RViz y los **tutoriales oficiales** siguen funcionando

El grafo

ROS, OpenCV y Eigen en el mismo grafo



Este grafo no cierra.

- `ros-kilted` (variante `desktop`) usa OpenCV 4.12. Mi nodo quiere la nueva 4.14
- En C++, dos OpenCV son **dos** `cv::Mat` en el mismo programa
- Conan lo detecta **al instalar**, antes de compilar una línea

El grafo · solución

Si no cierra, decides tú y Conan recompila

conanfile.py

```
class MiNodo(ConanFile):
    default_options = {"ros-kilted/*:variant": "desktop"}

    def requirements(self):
        self.requires("ros-kilted/2026.06.17")
        self.requires("opencv/4.14.0", force=True)
```

```
conan install conanfile.py --build=missing
```

- `force=True` : tu versión manda en todo el grafo, **también dentro de ROS**
- Lo que dependía de OpenCV se recompila: **un solo** `cv::Mat`
- Antes: compilar otra OpenCV dentro de la imagen y esperar que no choque

Rodeo 2 · Desarrollar

Para usar ROS, tu proyecto tiene que «hacerse ROS»

- La estructura del workspace: `src/ build/ install/ log/`
- `package.xml` y macros de `ament_cmake` dentro de tu CMake
- `colcon build` en lugar de tu build de siempre
- El IDE no encuentra nada si no hiciste `source` antes de abrirlo
- Librerías de terceros: unas de `apt`, otras copiadas al repo, otras a mano

> Si ROS es solo una capa de un producto C++ grande, el rodeo lo paga **todo** el producto.

Rodeo 2 · Desarrollar

Desarrollar dentro de la imagen

- Compilas con las herramientas que trae la imagen, que suele ser también la del robot
- El IDE no vive ahí: el índice, los breakpoints y el depurador cruzan el contenedor
- Volúmenes, permisos y `docker exec`: más pasos en cada ciclo de compilar y depurar

> Los dev containers ayudan mucho. Aun así, la imagen brilla más al desplegar que al desarrollar.

Sin rodeos

Un proyecto CMake normal que usa ROS

CMakeLists.txt

```
find_package(rclcpp REQUIRED)
find_package(OpenCV REQUIRED)

add_executable(my_node src/main.cpp)
target_link_libraries(my_node PRIVATE rclcpp::rclcpp opencv::opencv)
```

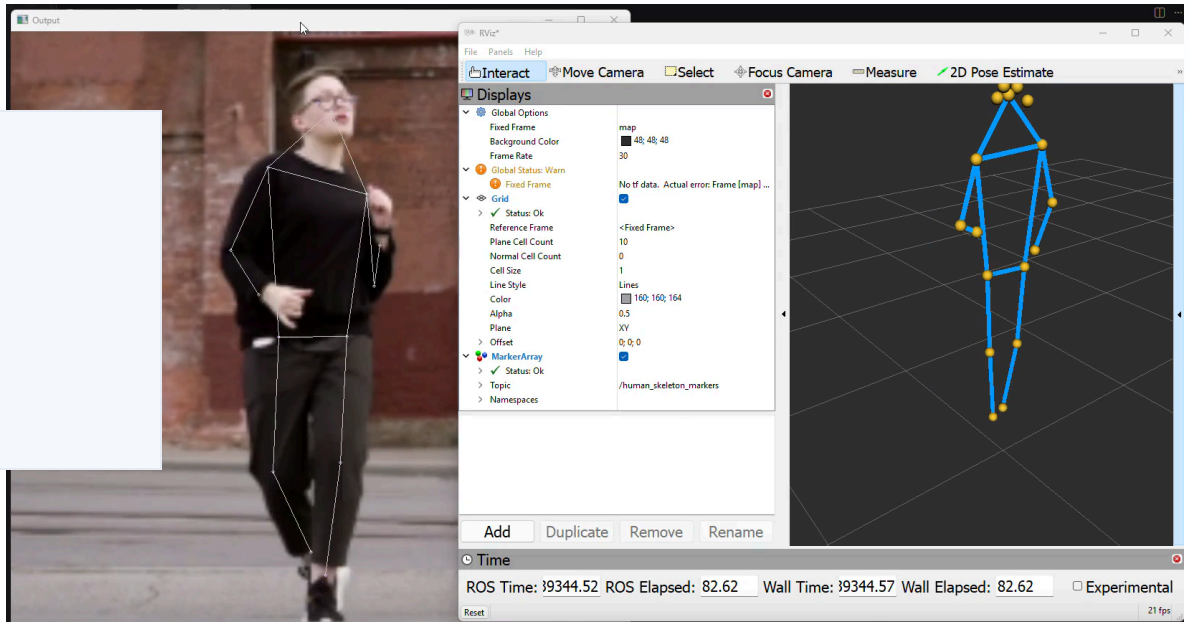
Instalar dependencias y compilar:

```
conan install conanfile.txt --build=missing
cmake --preset conan-release
cmake --build --preset conan-release
```

`Ejemplo real · examples/pose_estimation`

Estimación de pose humana: ROS + visión + ML

```
pose_estimation/
├── CMakeLists.txt
├── conanfile.txt
├── src/pose-estimation.cpp
└── assets/  # ML models
```



- Sin `package.xml`, sin `ament`, sin workspace.
- `ros-kilted`, `opencv` y `tensorflow-lite`, en un solo grafo.
- Todo se instala con un `conan install`

¿Ya tienes un workspace de colcon?

Conan dentro de tu workspace

mi_paquete/conanfile.txt

```
[requires]
tensorflow-lite/2.15.0

[generators]
CMakeDeps
CMakeToolchain
ROSEnv
```

```
conan install mi_paquete/conanfile.txt --build=missing
source /opt/ros/kilted/setup.bash       # tu ROS de siempre
source install/conan/conanrosenv.sh
colcon build
```

¿Ya tienes un workspace de colcon?

Qué hace ROSEnv

- Genera `conanrosenv.sh/bat`: colcon y CMake encuentran las librerías de Conan
- En tu `CMakeLists.txt`, nada específico de Conan: `find_package(tensorflowlite)`
- Funciona con **tu ROS de** `apt` o con `ros-kilted` **desde Conan**
- Sin tocar el sistema y sin copiar código de terceros al workspace

> No cambias tu flujo: es la puerta de entrada más sencilla.

Rodeo 3 · Desplegar

Llevar el código al robot

- **Compilar en el robot**: viable en una Jetson, muy lento en una placa modesta
- **Replicar el entorno de desarrollo en el robot**: herramientas y dependencias que el robot no necesita para funcionar
- **Copiar la carpeta** `install/` : rutas absolutas y librerías que faltan o sobran en el robot
- **¿Qué hay realmente instalado en el robot?** Difícil de contestar cuando lo pregunta un cliente, una auditoría o aparece un CVE

Sin rodeos

Compilar en tu máquina, ejecutar en el robot

robot_arm64 · perfil host

```
[settings]
os=Linux
arch=armv8
compiler=gcc
compiler.version=13
[tool_requires]
arm-gnu-toolchain/13.3
```

```
conan install conanfile.txt --profile:build=default --profile:host=robot_arm64
```

- **Build**: tu portátil, donde corren CMake y el compilador cruzado
- **Host**: el robot (ARM), al que van tu código y sus librerías

Sin rodeos

Llevar al robot solo lo que necesita

```
conan install conanfile.txt -pr:h=robot_arm64 --deployer=runtime_deploy --deployer-folder=robot
```

- Viaja solo el **runtime**: los ejecutables y las librerías dinámicas que tu nodo necesita para arrancar
- Se quedan en tu máquina las **herramientas de build**: CMake, compiladores, cabeceras, generadores
- Una carpeta: `scp` o una imagen mínima. El robot **no necesita Conan**

Sin rodeos

Qué va en cada robot: lockfile y SBOM

```
conan lock create . -pr:h=robot_arm64 --lockfile-out=robot.lock
conan install . -pr:h=robot_arm64 --lockfile=robot.lock   # genera sbom.cdx.json
conan audit list --sbom=sbom.cdx.json
```

- **El cliente** pregunta qué versión lleva su robot: el **lockfile** fija todo el grafo, hoy y dentro de un año
- **La auditoría** pide el **SBOM**: el inventario de todo el software del robot, en CycloneDX, generado con un hook en `conan install`
- **Aparece un CVE**: `conan audit` revisa ese SBOM contra vulnerabilidades conocidas

Volviendo a mi robot

Mi robot, sin rodeos



ROSCon Global 2026 · Toronto

Resumen

Los tres rodeos, simplificados

	Habitual	Conan
Instalar	El sistema, o una imagen por distro	Una dependencia más en la caché. Grafo consistente
Desarrollar	Workspace, `package.xml`, `ament`, `colcon`	CMake de siempre y `find_package`. O ROSEnv en tu workspace
Desplegar	Compilar en el robot, replicar el entorno o copiar `install/`	Perfiles build/host, `runtime_deploy`, lockfile y SBOM

Siendo honestos

Es una prueba de concepto

- **Receta monolítica**: toda la distro en un paquete. Cambiar una dependencia recompila ROS entero
- **Sin binarios precompilados**: el primer `conan install` compila ROS desde fuentes
- **Solo ROS Kilted**, con las variantes `core`, `base` y `desktop`
- **Sin probar a fondo**: interfaces propias (IDL) fuera de colcon, paquetes fuera de las variantes (Nav2, MoveIt...)

Pruébalo hoy

Necesitamos vuestro feedback

- Recetas y ejemplos en github.com/conan-io/ros-conan
- Probado en CI: **Linux** (x86_64 y arm64), **macOS** y **Windows**
- **Ejemplos**: CMake puro, workspace de colcon, pose_estimation (OpenCV + TensorFlow Lite)

Issues, ideas y críticas: **todas bienvenidas**

¡Gracias!

¿Preguntas?

Prueba los ejemplos:

- github.com/conan-io/ros-conan

Más info:

- *Bringing ROS into your native C/C++ workflow with Conan* - `discourse.openrobotics.org`
- *Robotics with Conan: consuming ROS as a regular package* - `blog.conan.io`

